নাগরিক সেবা

Nagorik Sheba

Software Requirements Specification

Version 0.2 — Week 2: Design & Planning

Project	Nagorik Sheba — Agentic Civic Complaint Platform
Document	Software Requirements Specification (SRS)
Version	v0.2
Sprint	Week 2 — Design & Planning
Course	Web Programming and Mobile Application Development
Institution	CSE Discipline, Khulna University, Khulna
Supervisor	Dr. Kazi Masudul Alam, Professor, CSE Discipline, Khulna University
Prepared by	Mozaza Al Zaman & Md. Tawfiqul Islam
Date	August 2026
Status	Week 2 Complete — Design Artifacts Finalized

Revision History

Version	Date	Description	Author
v0.1	Week 1	Initial SRS — scope, objectives, use cases, tech stack, team roles	Mozaza & Tawfiqul
v0.2	Week 2	Added Week 2 design deliverables — UI wireframes, ER diagram, system flowchart, use case refinement, GitHub Projects board configuration	Mozaza & Tawfiqul

Table of Contents

1. Introduction

1.1 Purpose of This Document

This Software Requirements Specification (SRS v0.2) is an updated version of the SRS v0.1 produced in Week 1. It extends the original requirements document with all design and planning deliverables completed during Week 2 of the project sprint. The additions cover the system architecture, database entity-relationship design, AI agent pipeline flowchart, UI/UX wireframes for both the citizen mobile app and the government staff dashboard, and the GitHub Projects board configuration for task allocation between team members.

This document serves as the authoritative design reference for the development sprints beginning in Week 3 and is submitted as a course deliverable for the Web Programming and Mobile Application Development course, CSE Discipline, Khulna University.

1.2 Project Overview

Nagorik Sheba (নাগরিক সেবা — "Citizen Service") is a multi-agent AI-powered civic complaint management platform built specifically for Bangladesh's three-tier local government structure — City Corporation, Pourashava, and Union Parishad. Citizens can report local problems including broken roads, water supply outages, gas leaks, electricity failures, and sanitation issues through a web application, mobile app, or SMS. Each complaint is automatically processed by a pipeline of five AI agents that verify the photo, classify the problem category, check for duplicate reports within a 300-metre GPS radius, calculate a priority score, and route the complaint to the correct Local Government Institution (LGI) based on GPS coordinates.

1.3 Scope

Version 0.2 of this SRS covers the following scope additions beyond v0.1:

- Complete UI/UX wireframes for all citizen mobile app screens and government staff dashboard screens
- Entity-Relationship (ER) diagram for the full 8-table database schema
- System flowchart illustrating the 5-agent AI complaint processing pipeline
- Refined use case diagram updated from v0.1 feedback
- GitHub Projects board configuration with task allocation for Weeks 3 through 6
- Architecture documentation linking all system components

1.4 Definitions and Abbreviations

LGI	Local Government Institution — City Corporation / Pourashava / Union Parishad
SRS	Software Requirements Specification
ER Diagram	Entity-Relationship Diagram — visual representation of database schema
AI Pipeline	5-agent automated complaint processing system
GPS	Global Positioning System — used for location-based routing

PWA	Progressive Web App — installable web app on Android
JWT	JSON Web Token — used for authentication
CRUD	Create, Read, Update, Delete — basic data operations
PR	Pull Request — peer code review mechanism on GitHub
OTP	One Time Password — used for citizen phone login

2. Week 2 Deliverables Summary

The following table summarises all deliverables completed during Week 2 of the project sprint. Detailed documentation for each deliverable is provided in the sections that follow.

Deliverable	Tool Used	Status
UI Wireframes — Citizen App	Figma + AI design tools	☑ Complete
UI Wireframes — Staff Dashboard	Figma + AI design tools	☑ Complete
ER Diagram — 8 tables	draw.io + Mermaid	☑ Complete
System Flowchart — 5-agent pipeline	Mermaid + draw.io	☑ Complete
Use Case Diagram (refined)	draw.io	☑ Complete
GitHub Projects Board	GitHub Projects	☑ Complete
Architecture Documentation	Markdown + diagrams	☑ Complete

3. UI/UX Wireframes

The UI wireframes were designed using Figma with AI-assisted design tools during Week 2. The wireframes cover all screens of the citizen mobile app (Progressive Web App) and the government staff web dashboard. The design follows a civic and professional visual identity using a primary dark teal color scheme for citizen screens and a dark navy sidebar for the government dashboard. Full Bangla language support (Hind Siliguri font) is included throughout.

3.1 Design Principles

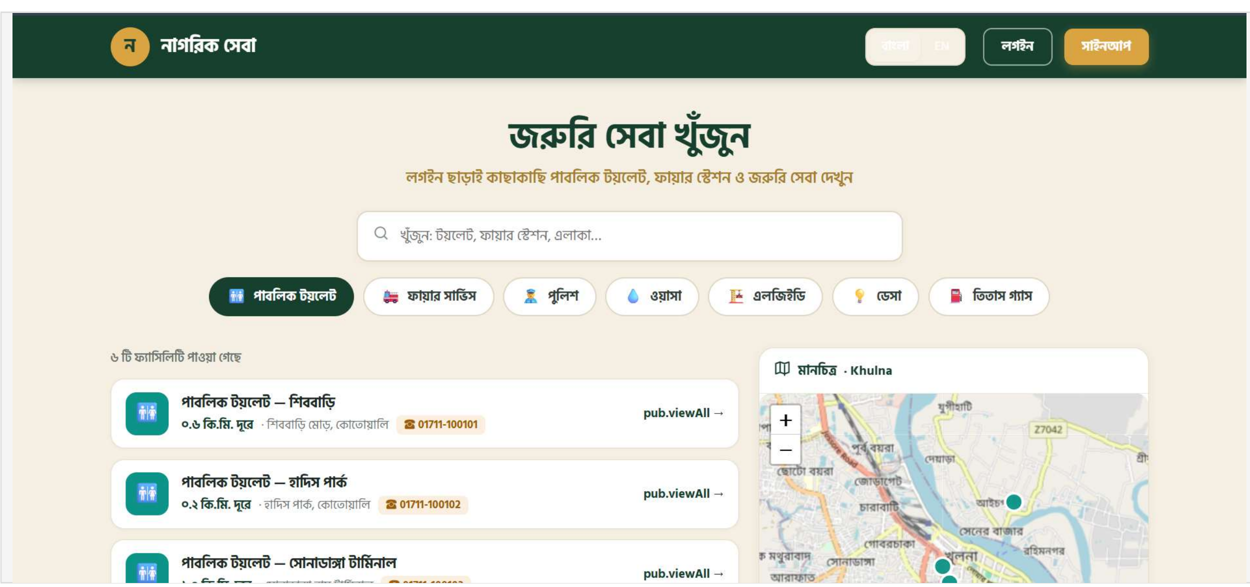
- Mobile-first responsive design — citizen app optimized for Android PWA installation
- Bangla and English bilingual UI throughout all screens
- Consistent status badge color coding — Open (yellow), In Progress (blue), Done (green), Rejected (red)
- Priority score shown as a colored circular badge — green (0–30), orange (31–60), red (61–100)
- All AI agent results displayed with a robot icon prefix to distinguish from human staff actions
- Skeleton screen loading states instead of spinners for low-bandwidth performance
- OpenStreetMap tiles with Bangla place name support for all map components

3.2 Citizen Mobile App Screens

3.2.1 Screen 1 — Nearby Facilities

Map view with toggleable facility type filters (Toilet, Fire Service, WASA, Police, LGED, DESA, Titas Gas), facility cards showing name, distance, contact number, and directions button.

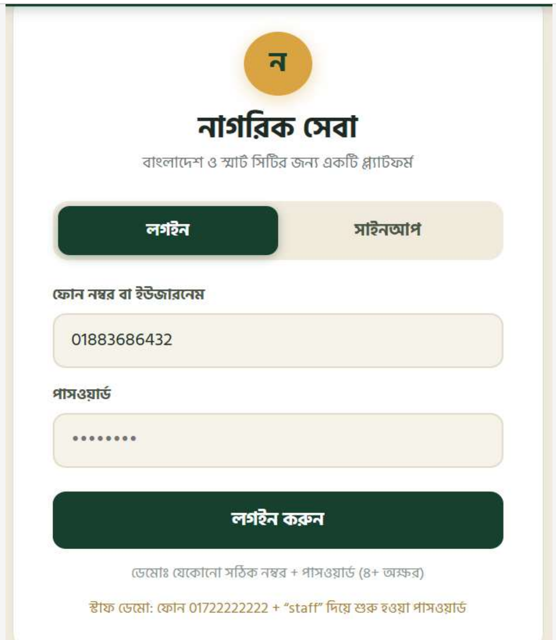
Insert: Nearby Facilities Screen Wireframe



3.2.2 Screen 2 — Splash and Login

Phone number input with OTP verification, register link, and NID verification notice. Logo centered on dark teal background.

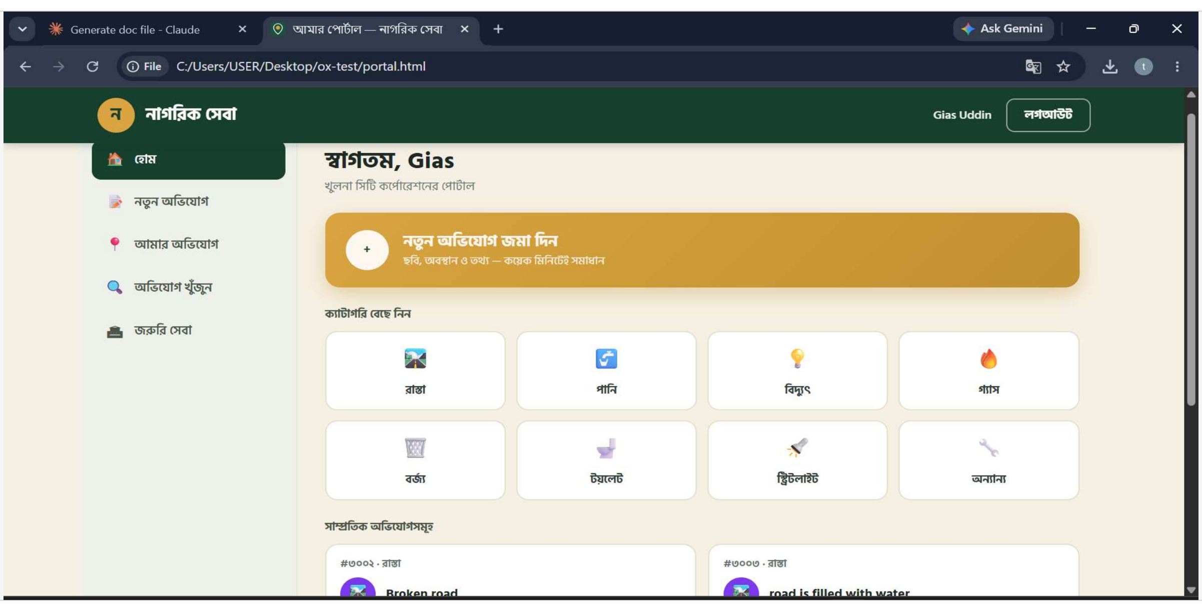
Insert: Login Screen Wireframe



3.2.3 Screen 3 — Home Feed

Live Leaflet.js map showing nearby complaint pins color-coded by priority, horizontally scrollable category filter row, and vertical complaint card list showing category icon, title, distance, vote count, time ago, and status badge.

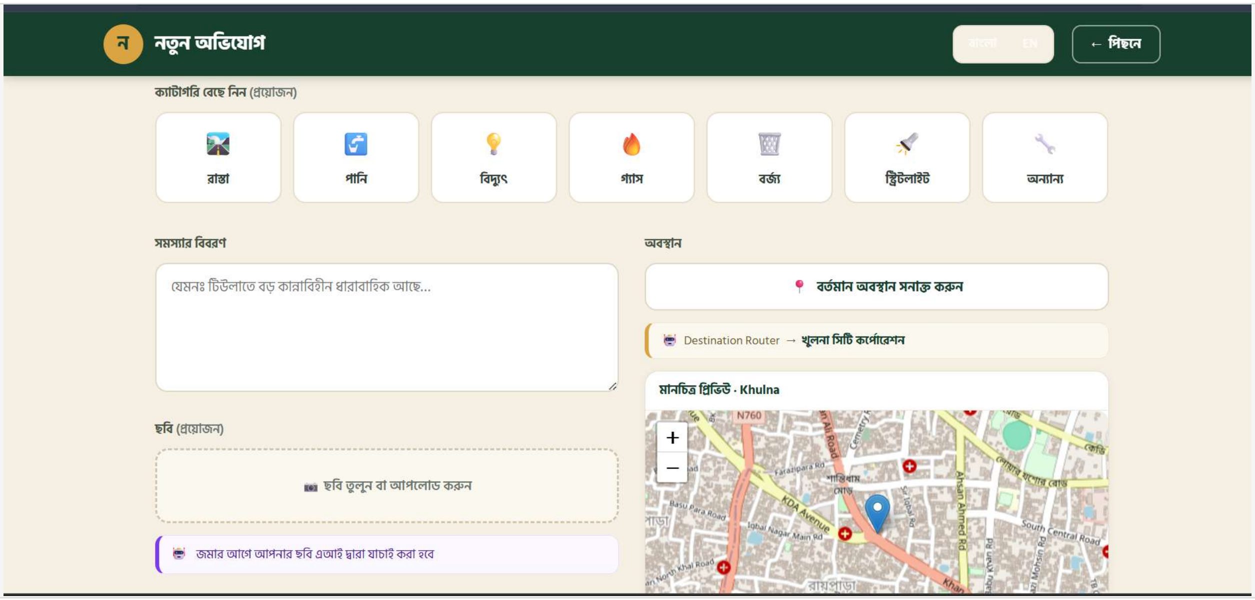
Insert: Home Feed Screen Wireframe



3.2.4 Screen 4 — Submit Complaint (4-Step Stepper)

Step 1: GPS location picker. Step 2: Category selector grid, title, and description. Step 3: Photo capture with AI verification notice. Step 4: Review summary and submit with estimated SLA time.

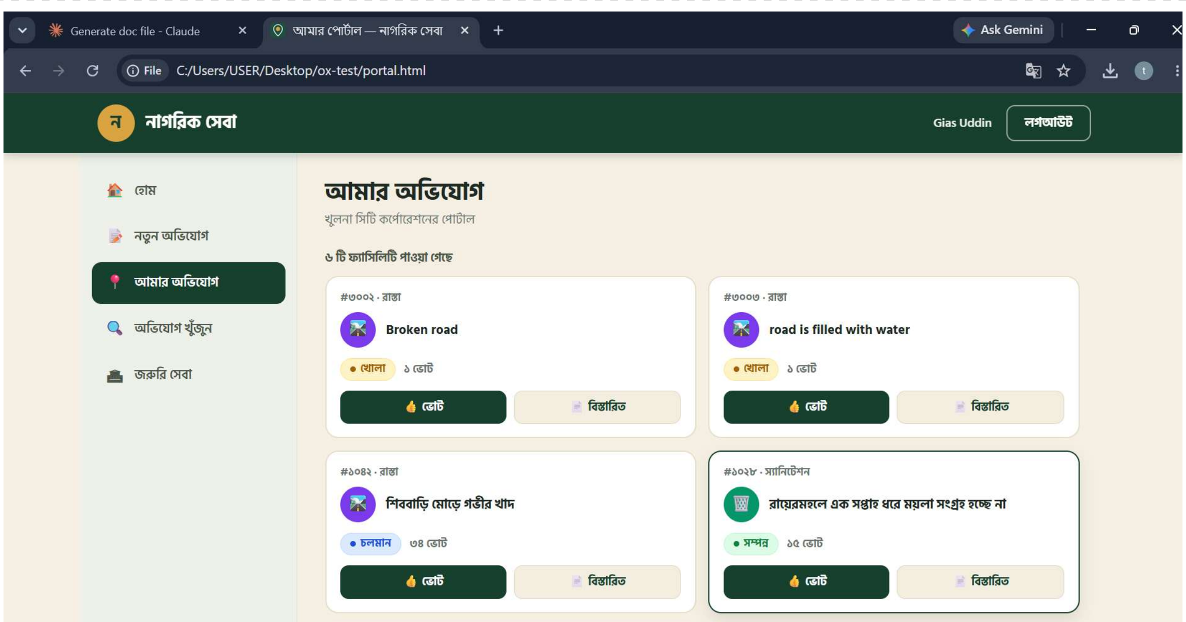
Insert: Submit Complaint Screen Wireframe



3.2.5 Screen 5 — My Complaints

Tabbed view (All, Open, In Progress, Done) with live AI pipeline progress bar per complaint, status badge, vote count, and pipeline stage tracker.

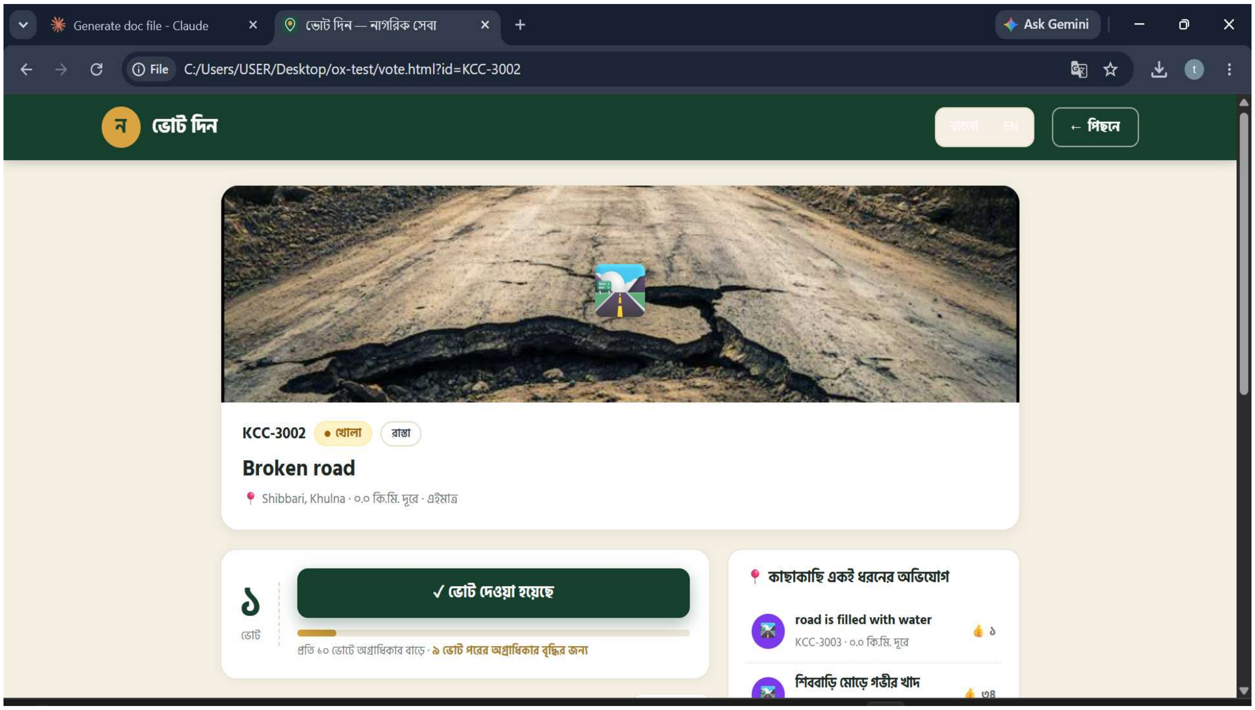
Insert: My Complaints Screen Wireframe



3.2.6 Screen 6 — Complaint Detail

Photo at top, full status timeline (Submitted → AI Verified → Classified → Prioritised → Routed → In Progress → Resolved), AI result summary card, vote button, and share button.

Insert: Complaint Detail Screen Wireframe

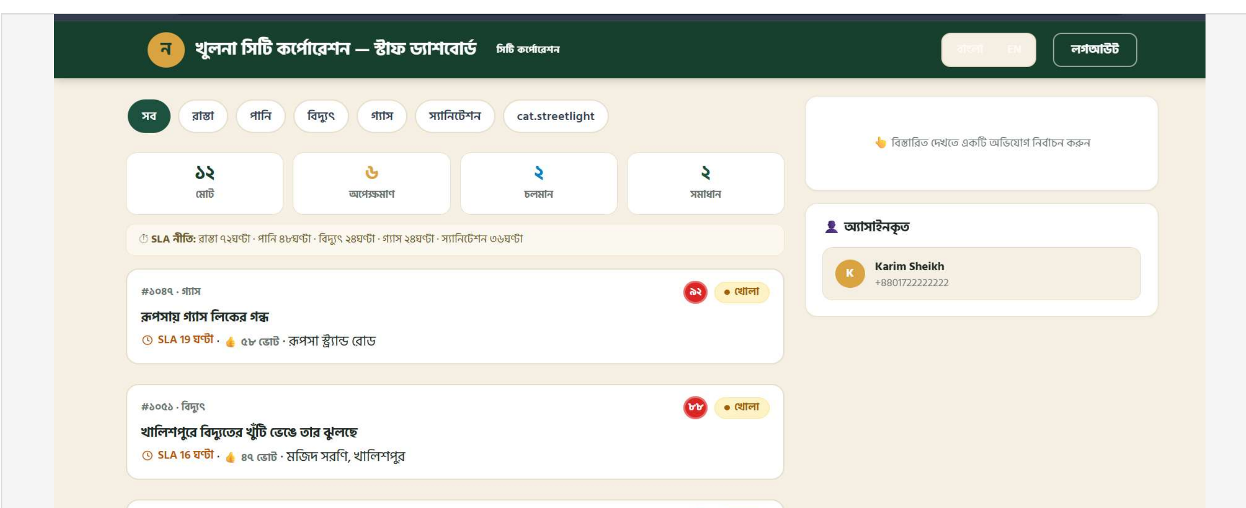


3.3 Government Staff Web Dashboard Screens

3.3.1 Screen 1 — Staff Login

Clean minimal login with LGI name displayed, email and password fields, and role indicator after login (Staff or Supervisor).

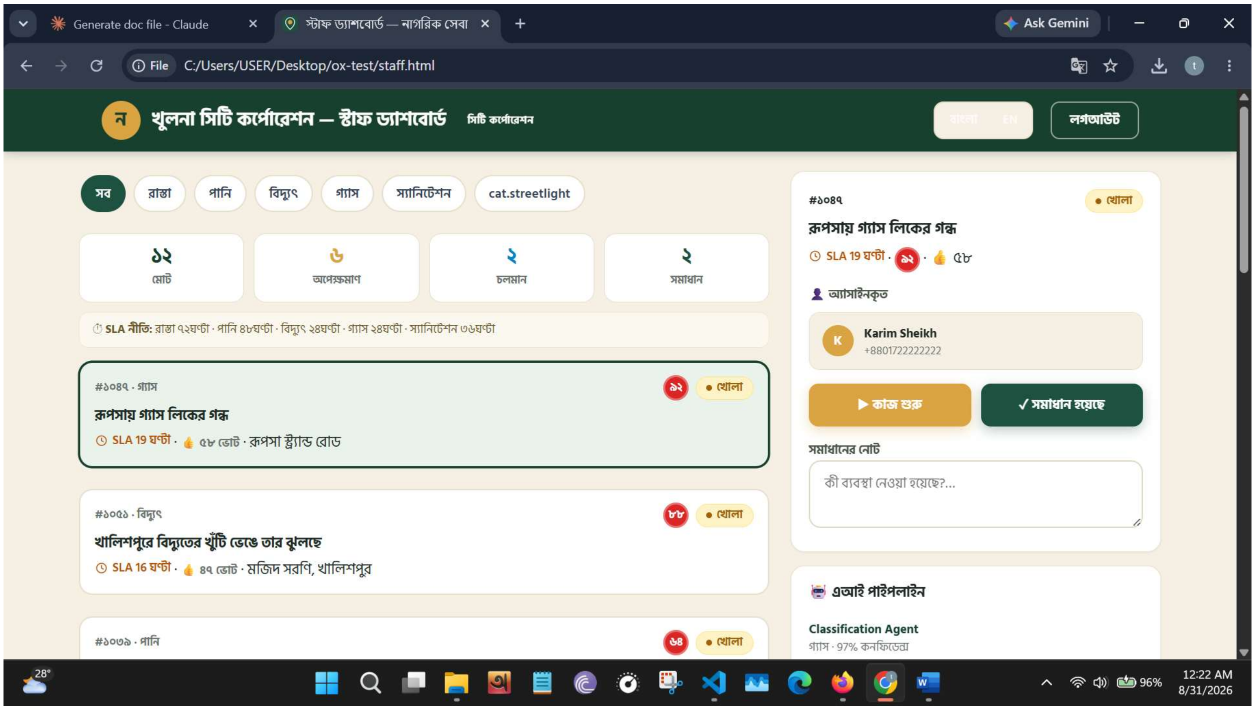
Insert: Staff Login Screen Wireframe



3.3.2 Screen 2 — Overview Dashboard

Left sidebar navigation with icons for Dashboard, Complaints, Map, Analytics, Facilities, Team, and Settings. Main area shows 4 stat cards (Open, High Priority, In Progress, Resolved This Month) and a sortable filterable complaint table with priority score, SLA remaining, and assignment controls.

Insert: Staff Dashboard Overview Wireframe



4. Database Architecture — Entity-Relationship Diagram

The database schema for Nagorik Sheba was designed during Week 2 and consists of 8 core tables. The ER diagram was produced using draw.io and Mermaid and covers all entities, their attributes, primary keys, foreign keys, unique constraints, and cardinality relationships.

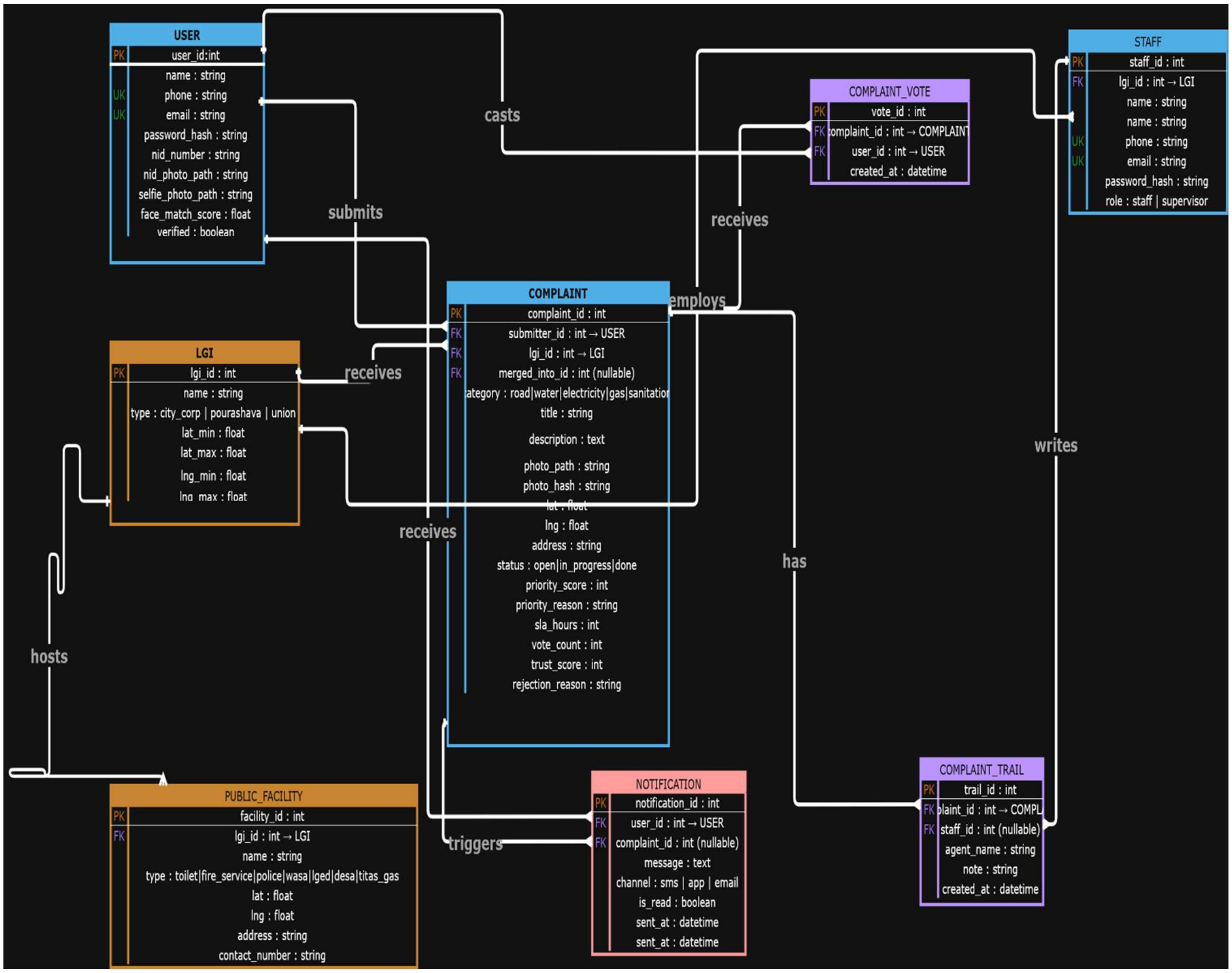
4.1 Tables Summary

Table	Key Fields	Purpose
USER	PK: user_id	Citizen accounts with NID verification and face match score
LGI	PK: lgi_id	Local Government Institutions with GPS bounding box boundaries
STAFF	PK: staff_id, FK: lgi_id	Government staff accounts linked to an LGI with role (staff/supervisor)
COMPLAINT	PK: complaint_id, FK: submitter_id, lgi_id, merged_into_id	Core complaint record — all pipeline outputs stored here
COMPLAINT_VOTE	PK: vote_id, FK: complaint_id, user_id	Citizen votes on complaints — unique per citizen per complaint
COMPLAINT_TRAIL	PK: trail_id, FK: complaint_id, staff_id (nullable)	Full audit log — null staff_id means AI agent wrote the entry
PUBLIC_FACILITY	PK: facility_id, FK: lgi_id	Public facilities (toilets, fire service, WASA, police etc) per LGI
NOTIFICATION	PK: notification_id, FK: user_id, complaint_id (nullable)	SMS and in-app notifications sent to citizens

4.2 Key Design Decisions

- COMPLAINT.merged_into_id is a self-referencing foreign key — duplicate complaints are linked to the original rather than deleted, preserving vote history
- COMPLAINT_TRAIL.staff_id is nullable — when null it indicates the entry was written by an AI agent; when populated it indicates a human staff action
- LGI.boundary is stored as four float fields (lat_min, lat_max, lng_min, lng_max) rather than a GeoJSON polygon for simplicity without requiring PostGIS
- COMPLAINT_VOTE has a UNIQUE(complaint_id, user_id) constraint to prevent duplicate voting
- NOTIFICATION.complaint_id is nullable — null allows general system messages not linked to a specific complaint

4.3 ER Diagram



5. System Architecture — Flowcharts

5.1 5-Agent AI Complaint Processing Pipeline

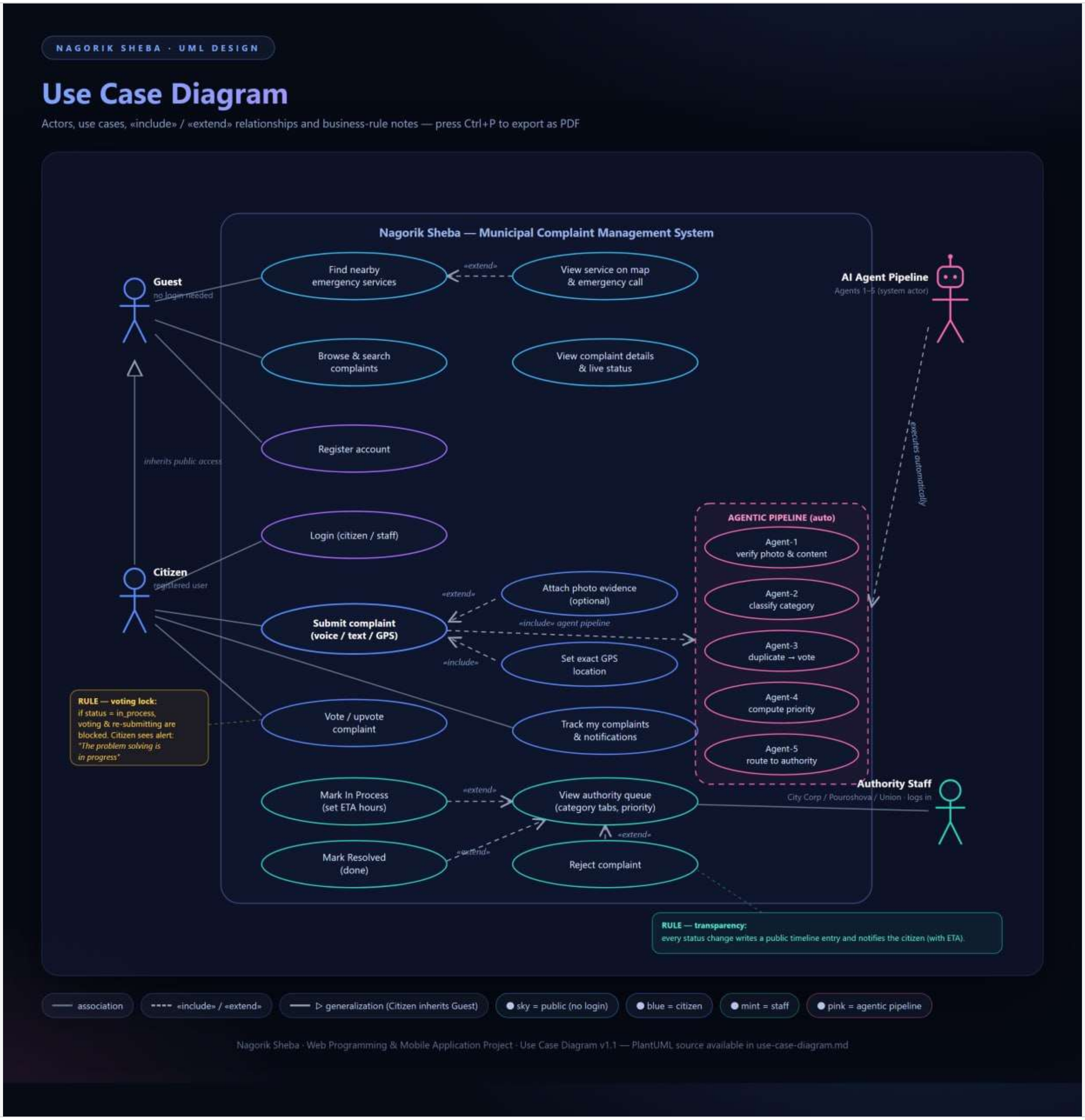
The following flowchart illustrates the complete automated journey of a complaint from citizen submission through the 5-agent AI pipeline to final routing to the responsible LGI staff dashboard. Every agent decision is logged to the COMPLAINT_TRAIL table with a null staff_id to indicate AI authorship.

5.1.1 Agent Descriptions

Agent 1 — Photo Verifier	Checks if the submitted photo is real, relevant, and not fake. Rejects spam, meme images, and irrelevant photos. Logs result to COMPLAINT_TRAIL.
Agent 2 — Classifier	Classifies the complaint into one of five categories: road, water, electricity, gas, or sanitation. Uses weighted Bangla and English keyword matching with 12/12 test cases passing.
Agent 3 — Duplicate Checker	Scans a 250-metre GPS radius for existing complaints in the same category. If a duplicate is found, the new submission is merged and the original's vote_count is incremented. The merged_into_id foreign key is set on the duplicate.
Agent 4 — Priority Ranker	Calculates a priority score based on: votes × 4 + severity keywords × 12 + age in hours. Stores priority_score and priority_reason in the COMPLAINT table.
Agent 5 — LGI Router	Matches the complaint GPS coordinates against LGI bounding boxes to determine whether the location falls under a City Corporation, Pourashava, or Union Parishad. Sets lgi_id on the COMPLAINT record.

5.3 Use Case Diagram

The use case diagram below shows the interactions between the three primary actors — Citizen, Government Staff, and Administrator — and the system features they can access.



6. GitHub Projects Board — Task Allocation

The GitHub Projects board was configured during Week 2 to manage task allocation between the two team members across all project sprints. The board follows a Kanban-style workflow with five columns.

6.1 Board Structure

📋 **Backlog**	All future tasks not yet started — Sprint 2 and beyond
🔜 **To Do**	Tasks planned for the current week, not yet started
🔄 **In Progress**	Tasks currently being actively worked on
👀 **In Review**	Completed tasks waiting for peer review via Pull Request
✅ **Done**	Fully completed and merged tasks

6.2 Task Allocation by Team Member

Mozaza Al Zaman	AI pipeline design and implementation, FastAPI/Node.js backend, ER diagram, system flowchart, GitHub Projects board configuration, deployment
Md. Tawfiqul Islam	Frontend development (React + Flutter), UI wireframes, database schema and seeding, use case diagram, pull request reviews

6.3 Labels Configured

- frontend — blue
- backend — green
- AI-pipeline — purple
- database — orange
- bug — red
- documentation — grey
- week-1, week-2, week-3-4, week-5-6 — yellow
- mozaza — teal
- tawfiqul — pink

6.4 Milestones Configured

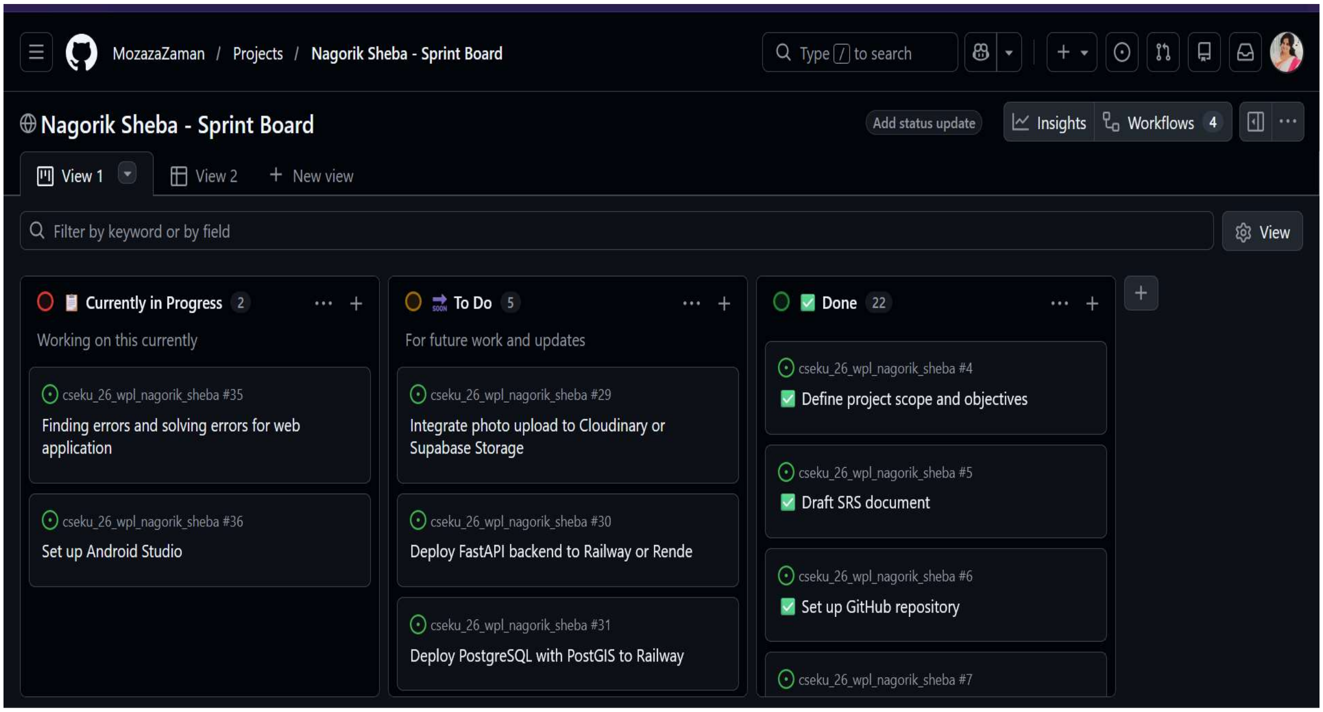
- Milestone 1: Week 3–4 Sprint 1 — Authentication, navigation, basic UI (due end of Week 4)
- Milestone 2: Week 5–6 Sprint 2 — CRUD operations, API integration, AI pipeline (due end of Week 6)

6.5 Branch Strategy

- main — protected branch, no direct push allowed

- dev/mozaza — Mozaza's active development branch
- dev/tawfiqul — Tawfiqul's active development branch
- All features merged to main via Pull Request with mandatory peer review and approval

6.6 GitHub Projects Board Screenshot



7. Updates from SRS v0.1

The following changes and additions were made in v0.2 relative to the original SRS v0.1:

Section	v0.1 Status	v0.2 Update
UI Wireframes	Not started — planned for Week 2	Complete — 8 citizen screens + 5 staff dashboard screens
ER Diagram	Not started — planned for Week 2	Complete — 8 tables with all relationships, draw.io + Mermaid
System Flowchart	Not started — planned for Week 2	Complete — 5-agent pipeline fully documented
Use Case Diagram	Draft version in v0.1	Refined and finalized with actor interactions
GitHub Board	Not configured	Fully configured — 76 tasks across 5 columns, 2 milestones, 9 labels
Database Schema	Described in text only	Fully designed — NOTIFICATION table added, self-ref FK for merges
Tech Stack	Node.js + SQLite + HTML5 planned	Confirmed and expanded — React 18 + Flutter + face-api.js

Section	v0.1 Status	v0.2 Update
LGI Routing	Described conceptually	Implemented with GPS bounding box per LGI (lat_min/max, lng_min/max)

8. Next Steps — Week 3–4 Sprint 1

With all Week 2 design artifacts completed and approved, the project proceeds to Development Sprint 1 covering Weeks 3 and 4. The following modules will be built:

8.1 Sprint 1 Objectives

- Set up PostgreSQL/SQLite database with all 8 tables and Alembic migrations
- Implement citizen registration with NID photo upload and face verification
- Implement phone OTP login for citizens and email/password login for staff
- Set up JWT token middleware and role-based access control
- Build React 18 frontend with Bangla font, bottom navigation, and basic screen routing
- Build Flutter mobile app with Material 3 theme and basic navigation
- Use GitHub Copilot for code scaffolding across all modules
- Generate unit tests for all authentication endpoints using pytest or Jest
- Conduct peer code review via Pull Requests on GitHub
- Produce Week 3 and Week 4 AI-generated progress summary reports

9. Acknowledgements

We express our sincere gratitude to Dr. Kazi Masudul Alam, Professor, CSE Discipline, Khulna University, Khulna, for his supervision, guidance, and constructive feedback throughout the design phase of this project. His critical evaluation during Week 1 review directly shaped the architectural decisions reflected in this document.

We also acknowledge the open-source tools and communities — draw.io, Mermaid, Figma, GitHub, OpenStreetMap, and the HuggingFace ecosystem — whose freely available platforms made the production of these design artifacts possible within a two-person university team context.